

LLD Case Study 16: Kafka Consumer Group Partition Rebalancing

The Bridge: How Do Consumers Share Work at Scale? In Chapter 16 of Track 1, we learned that Apache Kafka scales horizontally by splitting a topic into **Partitions** (e.g. 12 partitions).

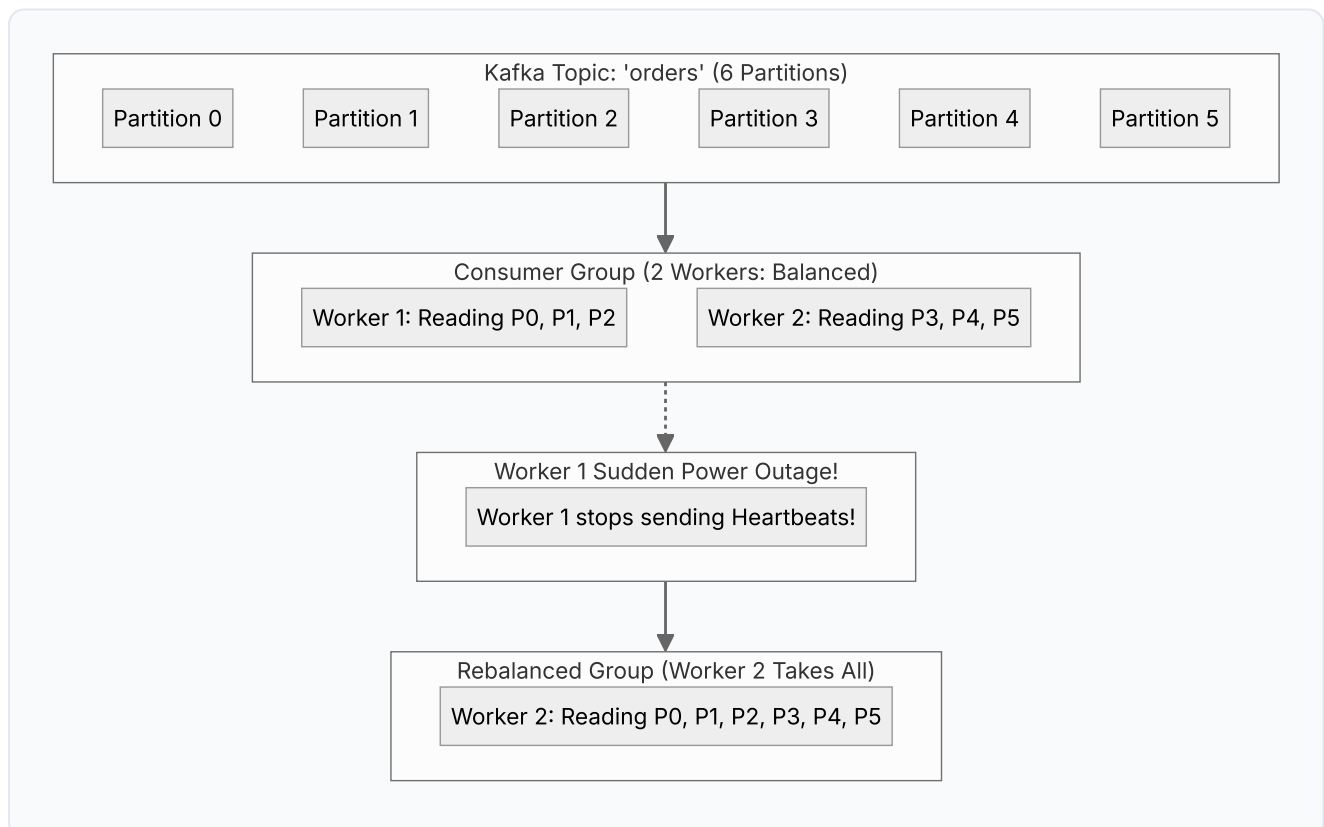
But what happens inside a **Consumer Group** when you have 4 worker applications reading those 12 partitions, and suddenly:

1. Worker 3 crashes!
2. Or you deploy 2 new servers to handle a traffic surge!

How do the surviving workers detect the crash, renegotiate ownership of the 12 partitions, and resume reading without duplicating or dropping messages?

This process is called **Consumer Group Partition Rebalancing**. In this case study, we will design the industrial **Cooperative Sticky Assignor** protocol and examine how modern streaming brokers avoid catastrophic "Stop-The-World" rebalance storms.

1. Zero-Prerequisite Intuition: The "Airport Baggage Carousel Shift"



The Eager vs Cooperative Rebalance Revolution

- **The Legacy Eager Way ("Evasive Stop-The-World"):** When Worker 1 died, Kafka forced ALL surviving workers to drop ALL their assigned partitions. The entire message processing pipeline stopped cold for 30 seconds while everyone renegotiated from zero!
 - **The Modern Cooperative Sticky Way (Incremental Handover):**
 - Surviving workers keep processing their existing partitions without interruption.
 - Only the orphaned partitions (from the dead worker) are reassigned to the surviving workers.
 - Downtime drops from 30 seconds to **0 seconds!**
-

2. Requirements & Architecture

Functional Requirements:

1. **Group Membership & Heartbeats:** Workers periodically send heartbeats to the Group Coordinator. If no heartbeat arrives within `session_timeout_ms`, the worker is marked dead.
 2. **Deterministic Partition Assignment:** Distribute M partitions across N active consumers fairly and evenly.
 3. **Sticky Allocation:** When rebalancing occurs, minimize partition movement (keep partitions on their previous consumers if still alive).
 4. **Generation ID (Epoch Fencing):** Every rebalance increments a `generation_id` to reject stale commits from zombie workers.
-

3. Production-Grade Python Implementation



4. Milestone Check: What You Just Mastered!

You now understand:

1. How Kafka divides topic partitions among consumer group members.
2. Why **Stop-The-World** rebalances caused multi-minute production outages in legacy systems.
3. How the **Cooperative Sticky Assignor** preserves live partition assignments while transferring only orphaned partitions.
4. How **Generation IDs** prevent stale writes from zombie workers that temporarily disconnected.

4. Edge Cases, Tests and Extensions

What the assignor guarantees and where it falls short

Property	Status	Detail
Even spread when members join or leave	Mostly	Shares differ by at most one in the common cases
Stickiness	Good	Members keep partitions they already hold, so only the necessary ones move
Balance when the partition count is not divisible	Bug	The retention cap is <code>target + 1</code> for every member whenever there is a remainder, so several members can keep the extra partition: 7 partitions split 3, 3, 1 instead of 3, 2, 2
Which partition is dropped when trimming	Arbitrary	<code>set.pop()</code> order is not deterministic across runs; sort for reproducibility
Failure detection	Good	A missed <code>session_timeout_sec</code> removes the member and triggers a rebalance
"Cooperative" in the class name	Overstated	A truly cooperative protocol revokes in a first round and assigns in a second, so unaffected consumers never stop; this simulation does one assignment step

Tests, including the imbalance bug

This block extends the implementation above. The last section replaces the retention cap with proper per-member quotas and checks the result on randomised scenarios.

Extensions interviewers ask for

1. **True cooperative rebalancing:** in round one the coordinator tells owners which partitions to revoke; in round two, after owners release them, the new owners receive them. Consumers that keep their partitions never pause.
2. **Static membership:** a restarting consumer with the same `group.instance.id` gets its old partitions back within the session timeout, avoiding a rebalance for a rolling restart.
3. **Generation fencing:** every commit carries the generation id; a commit from an older generation is rejected, which stops a zombie consumer from overwriting offsets after it was replaced.

4. **Rack awareness and weights:** extend the quota so heavier consumers receive proportionally more partitions, and prefer partitions in the consumer's zone.
5. **Rebalance storms:** flapping members cause repeated reassignments; add a minimum interval and a longer `session_timeout` than the longest GC or processing pause.

Follow-up questions

- *Why is stickiness valuable?* Moving a partition means committing offsets, closing readers and rebuilding local caches or state stores; keeping assignments stable keeps lag low during a rebalance.
- *What triggers a rebalance?* A member joining, leaving, or timing out, a change in the subscribed topics, or a change in partition count.
- *Heartbeat timeout versus processing time?* Heartbeats come from a separate thread, so a long batch does not by itself look like a failure; a separate `max.poll.interval` bounds how long processing may take between polls.